

Lab Exercise 9: Next Greater Element (NGE) using Stack

Objective: To write a Java program that maps the next larger element sequence for all array indices using a Monotonic Stack architecture.

Aim: To solve the NGE tracking loop challenge efficiently in $O(n)$ runtime rather than an $O(n^2)$ nested search layout.

Theory: A monotonic stack maintains elements in a strict order (increasing or decreasing). While parsing an array, elements are pushed onto the stack. When an incoming item exceeds the value at the top of the stack, it acts as the "next greater element" for those tracking indices, which are then popped off the stack.

Algorithm / Procedure:

1. Input: Array dimensions and its corresponding integer items.
2. Initialize an output array nge filled with -1 and an empty utility Stack.
3. Iterate across the elements. While the stack is not empty and the current element is larger than the item indexed at the top of the stack, pop the index and record the current item as its NGE.
4. Push the current index onto the stack and repeat until the array is fully processed.

Program Code: Problem5_NextGreaterElement.java

```
import java.util.*;
```

```
public class Problem5_NextGreaterElement {  
  
    public static int[] getNextGreaterElement(int[] arr) {  
  
        // TODO: Implement monotonic stack indexing routine here  
  
        return new int[arr.length]; // placeholder  
    }  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter array size: ");
```

```
int n = sc.nextInt();

int[] arr = new int[n];

for(int i=0; i<n; i++) arr[i] = sc.nextInt();


int[] result = getNextGreaterElement(arr);

System.out.println("NGE Result: " + Arrays.toString(result));

sc.close();

}

}
```

Sample Input / Output:

Input:

4

4 5 2 25

Output:

NGE Result: [5, 25, 25, -1]

Evaluation Criteria (Total Marks: 10):

- Program compiles and runs successfully (2 Marks)
- Proper implementation of Monotonic Stack operations (3 Marks)
- Correct array assignment and tracking logic (2 Marks)
- Handling edge cases, such as strictly decreasing arrays (2 Marks)
- Clean indentation and readable code structure (1 Mark)